

QUALITY ASSURANCE TESTING DOCUMENTATION
(QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	4
3. Test Environment & Execution Strategy	6
4. CI/CD Release Thresholds & Automation Gates	7
5. Test Case Specifications (Drafts)	8
6. Test Strategy & Plan Sign-Off	10

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - CATS (Context-Based Applicant Tracking System)
Team Repo URL	https://github.com/PremIshwar/Context-based-Applicant-Tracking-System/
Project Board URL	https://trello.com/invite/b/69ed095f06064b179d39487b/ATTI4cf35c43dcbc844a4328c3a55ff7fd25022A5C23/codekop-i-cats
Live Deployment URL (optional for preliminary round)	-

Objective:

The primary objective is to ensure that CATS can reliably extract structured data from resumes, trigger and execute targeted web scrapers (GitHub & LinkedIn), synthesize multi-source data into a unified JSON payload, produce accurate LLM-powered verification scores and candidate rankings via Z AI GLM, and auto-generate relevant interview questions all while maintaining acceptable response times, graceful failure handling when scrapers are blocked, and zero hallucination in evidence-based scoring. CI/CD checkpoints will enforce quality gates at every merge and deployment stage.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns testing connected back to specific user requirements via a "**Requirement Traceability Matrix**". It ensures every test has been conducted to prevent bugs/failure in products for that specific requirement and unplanned feature creep.*

1.1 In-Scope Core Features

- **Resume Ingestion & Parsing:** PDF/DOCX upload, text extraction, structured JSON output (name, skills, tenure, URLs), applicant review/edit step.
- **Job Description Parsing:** JD upload, extraction of required skills, preferred skills, experience level into structured JSON.
- **Digital Footprint Scraping:** GitHub API integration (commits, languages, repo activity), LinkedIn public data scraping (job titles, tenure), Google portfolio search.
- **Employer Dashboard:** Ranked candidate list, verification score breakdown, interview question view, "Unverified Profile" flagging, strengths, gaps, skills.

1.2 Out-of-Scope

- **Bulk Resume Batch Processing:** Individual or single ZIP upload only.
- **Interview Scheduling Integration:** No calendar booking or scheduling integration (example Calendly, Google Calendar).
- **Multi-Language Resume Support:** System only processes resume in English during the initial version.

2. Risk Assessment & Mitigation Strategy

Quality Assurance risks are anticipatory. Technical risks associated with CATS's application requirements and architecture are identified below, evaluated using the 5x5 Risk Assessment Matrix where Risk Score = Likelihood × Severity.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
<i>LinkedIn scraper blocked (403/429) during candidate evaluation.</i>	5	4	20 (Critical)	<i>Implement graceful fallback to resume-only evaluation. Cache known LinkedIn public page structures. Use multiple scraping approaches (direct HTTP vs. headless browser).</i>	<i>Simulate responses from LinkedIn scraper module. Verify fallback triggers correctly and "Unverified Profile" flag appears on dashboard. Monitor that no 500 error reaches the user.</i>
<i>LLM hallucinates evidence assigns verification score based on fabricated data not present in scraped payload.</i>	3	5	15 (High)	<i>Strict prompt engineering forcing citation of specific scraped data snippets. "No evidence = no score" rule. Post-LLM validation layer that cross-checks mentioned evidence against actual payload fields.</i>	<i>Feed LLM a resume claiming "Expert in Rust" with scraped GitHub data showing zero Rust repositories. Verify score reflects "Unproven" not fabricated proof.</i>
<i>Z.AI GLM API latency spike causing end-to-end pipeline to exceed 45-</i>	4	3	12 (Medium)	<i>Set timeout on LLM API call (30 seconds). Show "Processing..." indicator with estimated time.</i>	<i>Simulate GLM API response delay of 35 seconds. Verify timeout handling triggers. Verify UI</i>

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
<i>second threshold.</i>				<i>Implement async processing with polling for results.</i>	<i>shows appropriate loading state and eventually displays result or timeout error.</i>
<i>GitHub API rate limit exceeded (5,000 req/hr authenticated) during batch processing.</i>	3	3	9 (Medium)	<i>Implement request queuing with rate limit tracking. Cache GitHub data for 24 hours to avoid redundant calls. Limit concurrent evaluations to 5.</i>	<i>Simulate 50 concurrent candidate uploads. Monitor GitHub API call count and timing. Verify queuing activates and no 429 errors propagate to user.</i>

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and testing is required
16 – 25	Critical	Priority is Highest. Need an extensive level of testing.

3. Test Environment & Execution Strategy

- **Unit Test**

- *Scope: Resume text extraction logic, JSON schema validation (resume & JD), URL detection regex (GitHub/LinkedIn/Portfolio), Relevance Chunking keyword matcher, Verification Score calculator, and fallback trigger condition (empty URLs = skip scraping).*
- *Execution: Tests are written using PyTest. Execution happens locally during development AND automatically in the CI pipeline (GitHub Actions) on every code push to any branch.*
- *Isolation: All external dependencies are fully mocked. Z.AI GLM API calls are mocked with predefined JSON responses from fixture files. GitHub/LinkedIn scraper responses are mocked. This ensures unit tests only test internal logic, not external connections.*
- *Pass Condition: Happy cases pass (e.g., valid resume PDF → outputs correct JSON with name, skills, urls fields). Negative cases pass (e.g., .PNG file uploaded → returns error "Unsupported file type"). Edge cases pass (e.g., empty PDF → returns "No content extracted", resume without URLs → triggers fallback flag no_footprint: true).*

- **Integration Test**

- *Scope: Cross-component communication: Extractor → Scraper trigger, Scraper → Relevance Chunking → LLM payload assembly, LLM output → Dashboard data rendering, Fallback path (Scraper fails → Resume-only LLM path).*
- *Execution: Performed after Unit Testing passes on individual branches. Triggered when two or more service branches are merged (e.g., feature/extractor + feature/scraper merge into develop).*
- *Workflow: Real API calls are tested WITHOUT mocking where possible: Actual Z.AI GLM API calls with test payloads, actual GitHub API calls with a test account. LinkedIn scraping uses recorded fixtures only (because public scraping is too unstable for automated CI testing). Database/storage uses local SQLite or file-based JSON.*
- *Pass Condition: Integrated components function correctly together. Example: Valid resume upload → GitHub scraper fetches real data → chunking reduces payload → GLM returns valid Verification Score JSON → dashboard displays the correct score and evidence. No silent failures between components.*

- **Test Environment (CI/CD Practice):**

- *Local Testing: Manual testing on localhost (frontend) + localhost:3306 (backend). Developers test against local instances of all services.*

- **Regression Testing & Pass/Fail Rules:**

- *Local Testing: Manual testing on localhost:3000 (frontend) + localhost:8000 (backend). Developers test against local instances of all services.*
- *Staging/CI: GitHub Actions triggers on every push. Runs: (1) Linting (Ruff for Python, ESLint for TypeScript), (2) Unit Tests (PyTest), (3) Build verification (Vite build for frontend).*
- *CI/CD automated pipeline: Branch-specific push → triggers linting + unit tests only. Pull*

Request (PR) to main branch → triggers full PyTest suite + build verification. Merge to main → triggers deployment to staging environment for a final manual smoke test.

- **Test Data Strategy:**

- *Local Testing: Manual testing on localhost:3000 (frontend) + localhost:8000 (backend). Developers test against local instances of all services.*
- *Staging/CI: GitHub Actions triggers on every push. Runs: (1) Linting (Ruff for Python, ESLint for TypeScript), (2) Unit Tests (PyTest), (3) Build verification (Vite build for frontend).*
- *CI/CD automated pipeline: Branch-specific push → triggers linting + unit tests only. Pull Request (PR) to main branch → triggers full PyTest suite + build verification. Merge to main → triggers deployment to staging environment for a final manual smoke test.*

- **Passing Rate Threshold:**

- *To balance development velocity with system integrity, CATS adopts a tiered passing rate strategy. While minor, non-critical bugs are tolerated to avoid bottlenecking the hackathon timeline, any failure in core system logic or AI accuracy triggers an immediate pipeline block.*

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Failed
Automatic Build	<i>Zero Build Error</i>	<i>1 Error: GLM API error 504</i>	<i>Failed</i>
Unit Tests	<i>100% Passing Rate</i>	<i>100%</i>	<i>Passed</i>
Code Quality	<i>Zero Linting Error</i>	<i>97%</i>	<i>Failed</i>
Test Coverage	<i>Minimum 90%</i>	<i>80%</i>	<i>Failed</i>

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	98%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	78%	Failed
Critical Bugs	Zero P0/01 Bugs	1	Failed
API Performance	Response Time < 800ms	450ms	Passed
Security	API keys are not exposed	Clean	Passed

5. Test Case Specifications (Drafts)

- **1 Happy Case (Entire Flow):** This is the “Golden Path” where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Full Verification Pipeline	Verify recruiter uploads JD, applicant uploads resume and GitHub & LinkedIn URLs, system scrapes data, chunks it, sends to GLM, and produces a ranked	1. Employer registers. Employer uploads Backend Engineer JD. 2. Verify JD parsed into JSON. 3. Applicants register and put	JD parsed correctly. Resume parsed with all fields. Scrapers return valid data. Chunking reduces payload. GLM returns	JD parsed. GitHub returned list of project titles. LinkedIn was blocked. Chunking reduced payload 66%. GLM Score: 82%. Evidence

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		<i>candidate with Verification Score on dashboard.</i>	<i>the references URL (GitHub, LinkedIn). Applicants upload resume in PDF format. 4. GLM reviews and submits extracted JSON. 5. Verify GitHub & LinkedIn scrapers trigger. 6. Verify Relevance Chunking filters data. 7. Employer opens dashboard, sees candidate ranked with Verification Score. 8. Clicks candidate, sees evidence breakdown & suggest interview questions.</i>	<i>Verification Score (e.g., 82%) with evidence citations. Dashboard shows ranked candidate. Pipeline completes in <45s.</i>	<i>breakdown correct. 4 questions generated. Status: Passed</i>
TC-02	Specific Case (Negative): No Digital Footprint	Verify system handles candidate with zero URLs gracefully no scrapers trigger, fallback to resume-only matching, "Unverified Profile" flag displayed, no 500 errors.	1. JD already uploaded. 2. Applicant didn't provide a relevant URL. 3. System parses resume. 4. Verify fallback path activates. 5. Employer opens dashboard. 6. Applicant scores lower. 7. Verify questions are generic.	Resume parsed with empty URLs. Zero scraper calls. Fallback triggers. Lower applicant score. Scored low. 3 generic questions generated. No errors.	Resume parsed, urls: []. Zero scraper calls logged. Fallback triggered. Badge displayed red. Scored low. 3 generic questions generated. No errors. Status: Passed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-03	NFR (Performance): E2E Pipeline Latency	Verify complete pipeline (upload → extract → scrape → chunk → LLM → dashboard) completes within 45 seconds per candidate under normal load.	1. JD pre-loaded. 2. Upload test resume and portfolio links 3. Record pipeline duration for each. 4. Monitor for timeout errors.	Zero timeout errors across 5 runs	Processed successfully. Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"You are The Curious Critic, an expert interview question designer for a candidate screening system.	JSON file with questions, context based	You mentioned zero-day mitigation at OWASP KL. What specific techniques do you advocate for?	Pass
AI-02	"You are The Jumbled Jsoneer, a precise	{ "cert": ["Certified Red	{ "cert": ["Certified Red	Pass

	information-extraction agent.	Team Practitioner (CRTP)", "Practical Ethical Hacking", "Google Threat Intelligence", "Linux Forensics", "Deep Dive into Network As...	Team Practitioner (CRTP)", "Practical Ethical Hacking", "Google Threat Intelligence", "Linux Forensics", "Deep Dive into Network As...	
AI-03	You are The Lexical Librarian, a meticulous skill-verification agent for a candidate screening system.	Certifications 95	Certifications 95	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	Input is limited by page count and file size of PDF.
Input used while testing	Multiple page resumes uploaded.
Expected Behavior	Text is extracted from the resume and sent to JSONeer agent for structuring.
Actual Behavior	PDF is successfully processed and response is stored in database.
Status	Pass

6.3. Adversarial/Edge Prompt Test

Ambiguous Input/Edge Prompt Test:

```
{
  "cert": [
    "Certified Red Team Practitioner (CRTP)",
    "Practical Ethical Hacking",
    "Google Threat Intelligence",
    "Linux Forensics",
    "Deep Dive into Network Assessments",
    "Fundamentals of Network Concepts, Protocols and Security",
    "Hands-on Network Security",
    "Implementing Secure Network Function Virtualization",
    "Introduction to Wireless Security",
    "Mastering Network Security",
    "Burp Suite: Web Application Penetration Testing",
    "CVEs for Ethical Hacking, Bug Bounties & Penetration Testing",
    "Top 5 Tools & Tricks for Ethical Hacking & Bug Bounties",
    "Digital Forensics Essentials (DFE)",
    "Network Defence Essentials (NDE)"
  ],
  "github": [
    {
      "title": "andrew-kramer.github.io",
      "readme": ""
    }
  ]
}
```

GLM Output:

The applicant is a highly motivated cybersecurity undergraduate with an impressive certification portfolio—including the rigorous CRTP—and notable CTF achievements, demonstrating strong offensive security knowledge. However, they are fundamentally mismatched for the Senior Cybersecurity Analyst role. They possess zero full-time professional cybersecurity experience against the 5 years required (SOC Analyst and Network Administrator), are currently completing their bachelor's degree, and lack critical senior-level defensive skills including SIEM Tools, Incident Response, and Firewall Management. While their certifications and competitive drive are commendable for a junior or entry-level position, the experience and operational maturity gap is too significant for a senior role requiring on-call incident response and enterprise security oversight.

6.4. Hallucinating Handling

In CATS using Ilmu GLM model with task-specific agents, hallucination is mitigated by forcing every decision to be grounded in verified inputs like structured resumes, job descriptions, and predefined scoring rubrics rather than open-ended reasoning. Each agent is narrowly scoped (e.g., parsing, matching, scoring, explaining), which reduces ambiguity and prevents the model from inventing information outside its task. Outputs are constrained to strict schemas (such as JSON) and must include explicit evidence references tied directly to source data, ensuring that no claim can exist without traceable support. A dedicated validation or “critic” agent reviews all outputs to detect unsupported statements, inconsistencies, or vague reasoning, rejecting or regenerating responses when necessary. Retrieval mechanisms further limit hallucination by ensuring the model only operates on relevant, pre-fetched context instead of relying on its internal knowledge. Confidence scoring is added to quantify uncertainty, allowing low-confidence outputs to be flagged for human review. Together, this layered design shifts the system from generative guessing to controlled, auditable decision-making.